Setup Guide

Run the AI Gateway Assessment on your own laptop

Good news: you do not need any API keys, accounts or paid services. Every AI model and tool server in this project is a pretend (mock) version. You only need Python and an internet connection for the first install.

Tested on: Windows 11 with Python 3.14. The commands for macOS and Linux are included but were not run by the author. Commands in this guide are for **Windows PowerShell** unless a step says otherwise.

Before you start: what you need

Tool	Version	How to check
Python	3.11 or newer	python --version
Internet	first install only	(to download packages)
Node.js (optional)	any recent version	node --version. Only for the MCP Inspector in step 6.

Python download: python.org/downloads. On Windows, tick **Add python.exe to PATH** in the installer.

1 Unzip and open a terminal in the project folder

Unzip the file. Inside you should see **pyproject.toml**, **README.md** and the folders **task1_mcp_server**, **task2_mcp_gateway**, **task3_llm_guardrail** and **task4_model_router**. Open a terminal **inside that folder**: right-click the folder and choose *Open in Terminal* (Windows), or use `cd` on macOS and Linux.

Important: every command in this guide must be run from this folder. Most errors come from being one folder too high, or from forgetting step 3.

2 Create the virtual environment and install

A virtual environment keeps this project's packages separate from the rest of your computer. This takes 1 to 3 minutes.

```
# Windows (PowerShell)
python -m venv .venv
.venv\Scripts\python -m pip install -e ".[dev]"

# macOS / Linux
python3 -m venv .venv
.venv/bin/python -m pip install -e ".[dev]"
```

If PowerShell says *running scripts is disabled*, run this once and try again: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

3 Activate the environment

```
# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate
```

Your prompt should now start with **(.venv)**. **Every new terminal** needs this step again, after moving into the project folder.

4 Check that it works: run the automated tests

```
python -m pytest
```

Expected last line: **248 passed** (about 40 seconds). The first run can take a few seconds longer because the token counter downloads a small vocabulary file; if you are offline it automatically uses an estimate instead.

To test one task only: `python -m pytest task1_mcp_server` (also task2_mcp_gateway, task3_llm_guardrail, task4_model_router).

5 Try Task 1 by hand (the MCP server)

```
python try_task1.py
```

It starts the server and calls both tools with good and bad input. You should see 12 result lines, for example:

```
valid customer lookup     -> OK: {... "Ada Lovelace" ...}
customer not found        -> TOOL ERROR (isError): customer_not_found
bad id format (lowercase) -> JSON-RPC ERROR -32602
unknown tool              -> JSON-RPC ERROR -32602
```

6 Try Tasks 2, 3 and 4: start everything and run the demo (Windows)

```
.\start.ps1     # starts 7 services in the background, prints OK for each
.\demo.ps1      # runs the demo of all three gateways
.\stop.ps1      # stops everything when you are done
```

What a successful demo shows:

- **Task 2:** a viewer calling admin_reset_key gets **-32001 Unauthorized Tool Call**; an admin succeeds.
- **Task 3:** four lines of text arrive about 0.3 seconds apart, with **[REDACTED]** where the email, SSN and card number were.
- **Task 4:** a normal request is served by **primary**. With the main model forced slow it is served by **secondary** after about 3 seconds. Big requests then get 200 followed by **429** with a retry-after value.

If a service fails to start, open the file `.run\<service-name>.err.log` to see why.

Optional: Task 1 in the MCP Inspector (needs Node.js)

```
npx @modelcontextprotocol/inspector python -m task1_mcp_server.server
```

This opens a web page where you can click through the two tools. It is optional because the tests and try_task1.py already check the same thing.

macOS and Linux: starting Tasks 2, 3 and 4 by hand

The .ps1 scripts are Windows only. Run each line below in its own terminal (activate the environment in each one). These commands were not run on macOS or Linux by the author.

```
# Task 2 - MCP gateway
python -m uvicorn task2_mcp_gateway.mock_downstream:app --port 9001
JWT_SECRET=demo-secret-demo-secret-demo-secret-123 DOWNSTREAM_URL=http://127.0.0.1:9001/mcp pytho

# Task 3 - streaming PII guardrail
MOCK_LLM_DELAY_MS=300 python -m task3_llm_guardrail.mock_llm
python -m task3_llm_guardrail.app

# Task 4 - rate limiter and router
python -m uvicorn task4_model_router.mock_providers:primary_app --port 9301
python -m uvicorn task4_model_router.mock_providers:secondary_app --port 9302
python -m task4_model_router.app
```

Then use the curl examples in README.md.

Which service uses which port

Port	Service	Part of
9000	MCP gateway (security door)	Task 2
9001	Mock MCP server	Task 2
9100	LLM gateway (privacy filter)	Task 3
9101	Mock LLM	Task 3
9300	Model router (limits and backup)	Task 4
9301 / 9302	Mock primary / secondary model	Task 4

Opening an address like http://127.0.0.1:9000/ in a browser shows *Not Found*. That is normal: the real endpoints only accept POST requests, and it shows the service is running. To see a running service's API page, open `http://127.0.0.1:9000/docs` (or port 9100 or 9300).

Common problems and fixes

Message you see	Cause and fix
No module named 'task1_mcp_server'	Wrong folder, or the environment is not active. cd into the project folder and run step 3.
does not appear to be a Python project	You are not in the folder that contains pyproject.toml.
npx is not recognized	Node.js is not installed. It is only needed for the optional Inspector.
running scripts is disabled	Run once: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

Message you see	Cause and fix
address already in use	An earlier run is still going. Run .\stop.ps1 or close that terminal.
A service shows FAILED in start.ps1	Read .run\<service-name>.err.log for the reason.
Task 1 server prints one line and waits	Normal. It waits for an MCP client. Press Ctrl+C to stop it.

Where to read more

- **README.md**: overview, architecture diagram and run commands for each task.
- **task1_mcp_server/NOTES.md** to **task4_model_router/NOTES.md**: design decisions and limitations of each task.